Your Presentation Title

Your Subtitle Here

Your Name
Your Institution

outline

- first topic — **key point**
- second topic — **key point**
- third topic — **key point**
- **conclusion and future work**

slide title

- first point
- second point
- third point

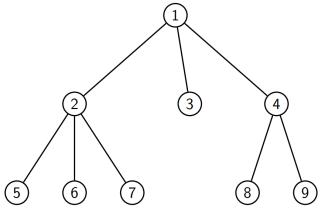
first line of centered text

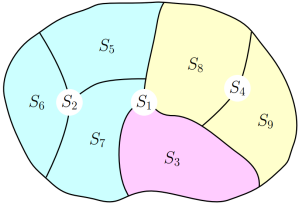
emphasized line

highlighted conclusion

Hierarchical clustering

partitionings of data points with varying degrees of "granularity"



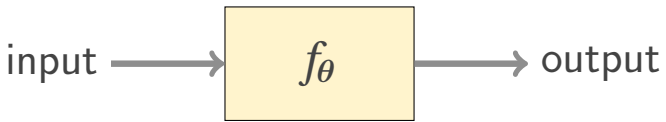


J. Chen, H. Avron, and V. Sindhwani, "Hierarchically compositional kernels for scalable nonparametric learning," *Journal of Machine Learning Research*, vol. 18, no. 66, pp. 1–42, 2017.



diagram caption

flow diagram



two column layout

Left column

- point one
- point two

Right column

- point one
- point two

mathematical content

Given a dataset $\mathcal{D} = \{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=1}^N$, we aim to:

$$\min_{\boldsymbol{\theta}} \frac{1}{N} \sum_{i=1}^{N} \mathcal{L}(f_{\boldsymbol{\theta}}(\mathbf{x}_i), \mathbf{y}_i)$$

where:

- f_θ is the model with parameters θ
- $\mathcal{L}$ is the loss function

an equation and its reading

$$\min_{\boldsymbol{\theta}} \frac{1}{N} \sum_{i=1}^{N} \mathcal{L}\big(f_{\boldsymbol{\theta}}(\mathbf{x}_i), \mathbf{y}_i\big)$$

f_θ the model, $\mathcal{L}$ the loss

code example

```
import torch
import torch.nn as nn

class Model(nn.Module):
    def __init__(self, input_dim, output_dim):
        super().__init__()
        self.linear = nn.Linear(input_dim, output_dim)

    def forward(self, x):
        return self.linear(x)
```

contributions

- first contribution
- second contribution
- third contribution